

Discussion / Analysis

Overloading Operations Across Set Types

Since bool arrays and int arrays are different types, you can overload functions to handle both. If you pass in a bool array the compiler picks the set version, if you pass in an int array it picks the multiset version. The return types can differ too if needed.

Using a Single Structure for Both Types

You could store both in an int array and just check if the value is nonzero for set operations. The downside is you lose the ability to overload since there is no longer a type difference to distinguish them.

Inferring Set Type from Natural-Language Queries

Pretty difficult. Word choice gives hints, "which customers" points to a set and "how many customers" points to a multiset, but users do not always phrase things clearly so you would need real parsing work to get it right consistently.

Converting Between Set Types

Going from multiset to set is fine, you just drop the counts. Going from set to multiset is technically possible but you end up with counts of only 0 or 1 which does not give you anything useful.

Implications for a Natural-Language Query Interface

Since you cannot always tell what the user wants from the query alone, you would need to either ask for clarification or default to multiset since it is the more general option and can be downgraded if needed.

Implications for Arbitrary Data Access

Adding more types like fuzzy sets or ordered collections makes the problem harder. The more types you support the more ways a query can be misread, and there is no clean way to map between all of them automatically.